StreetRace Manager
Phase 2: Integration Testing Report

Student Roll Number: 2024101116

March 23, 2026

1 Overview

This report documents Phase 2 integration testing for the command-line system **StreetRace Manager**. The objective is to verify that independent modules interact correctly under realistic workflows and that business rules are enforced across module boundaries.

1.1 Implemented Modules

Required modules:

- Registration Module
- Crew Management Module
- Inventory Module
- Race Management Module
- Results Module
- Mission Planning Module

Additional custom modules (2):

- Betting Module: allows racers to place bets before a race and processes payout after race result.
- Reputation Module: updates racer reputation after race outcomes and supports eligibility checks.

2 Integration Scope and Goals

The integration phase validates data flow, control flow, and rule consistency between modules rather than isolated unit behavior. The main goals are:

- Confirm end-to-end flows across registration, race participation, result processing, and post-race side effects.
- Confirm business constraints across modules (role/eligibility/resource checks).
- Detect interaction defects caused by missing setup dependencies.

3 Business Rules and Cross-Module Enforcement

The system is evaluated against assignment-level rules and equivalent implemented constraints:

1. A racer must exist in Registration before downstream operations (crew, race, missions, betting, reputation).
2. Race registration requires:
   - existing race,
   - existing racer,
   - sufficient racer balance for entry fee,
   - inventory speed validation for the active car.
3. Race result processing awards prize money to winner and notifies mission tracking.
4. Mission completion awards bonus to registered racer.
5. Betting payout only occurs for correct winner prediction.
6. Reputation updates are based on participants list and winner identity.

4 Call Graph

4.1 Submission Requirement

Hand-Drawn Call Graph Image:

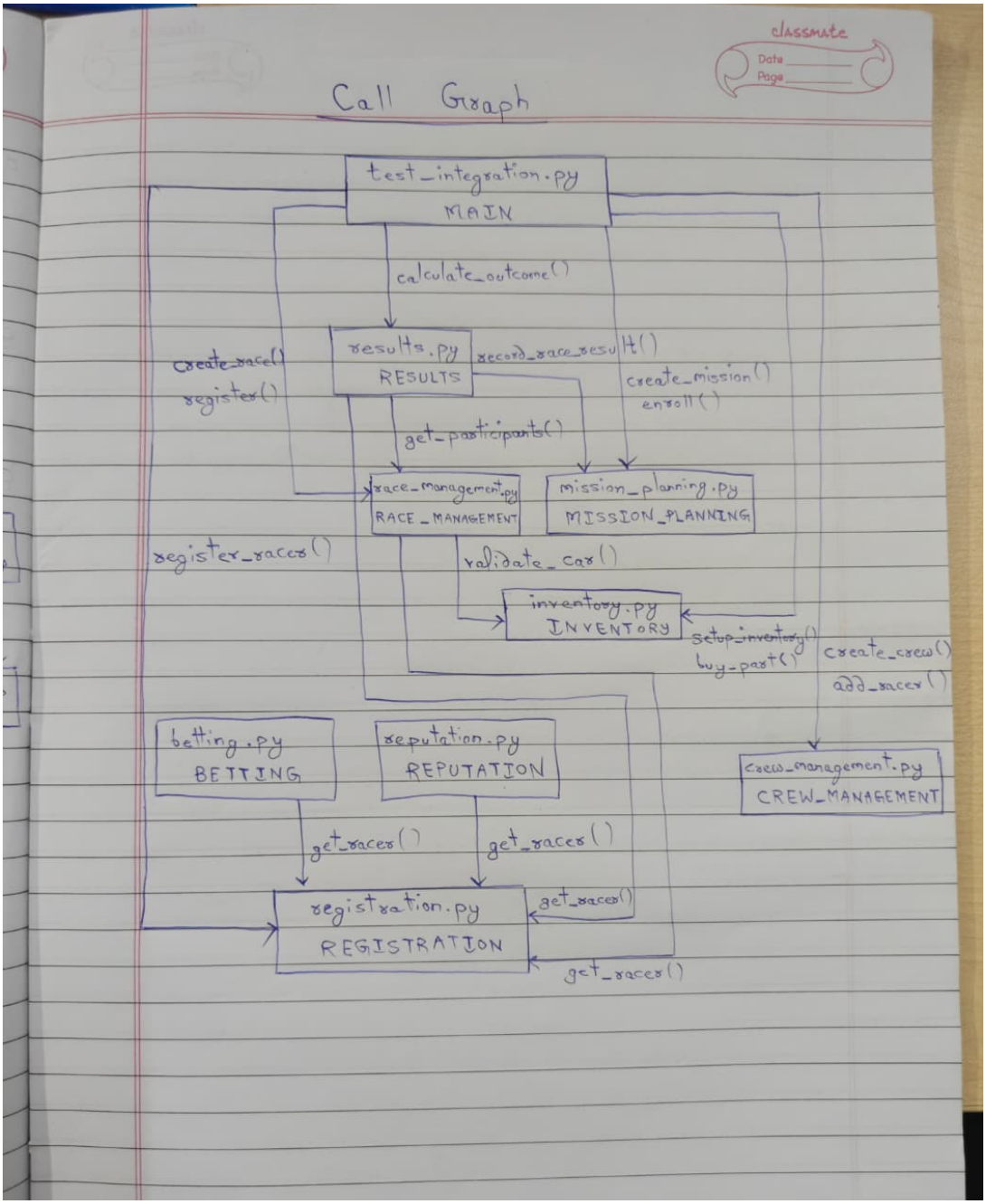

Figure 1: Hand-drawn call graph for StreetRace Manager integration

4.2 Function-Level Call Relationships (for drawing guidance)

Use each function below as a node and draw arrows from caller to callee.

Core integration paths:

- `RaceManagementModule.register_for_race` → `RegistrationModule.get_racer`
- `RaceManagementModule.register_for_race` → `InventoryModule.validate_car_requirements`
- `ResultsModule.calculate_race_outcome` → `MissionPlanningModule.record_race_result`
- `MissionPlanningModule.record_race_result` → `RegistrationModule.get_racer`

Additional integration paths used by tests:

- `CrewManagementModule.add_racer_to_crew` → `CrewManagementModule.get_crew`
- `BettingModule.place_bet` → `RegistrationModule.get_racer`
- `BettingModule.process_race_results` → `RegistrationModule.get_racer`
- `ReputationModule.get_reputation` → `RegistrationModule.get_racer`
- `ReputationModule.process_race_results` → `ReputationModule.get_reputation`

5 Integration Test Design

5.1 Test Environment

- Language: Python 3.12
- Framework: pytest
- Test file: `integration/tests/test_integration.py`
- Execution command: `python -m pytest integration/tests/test_integration.py -q`

5.2 Designed and Executed Test Cases

ID	Scenario Tested	Modules Involved	Expected Result	Actual Result	Errors / Issues Found
T1	Register racers and add them into one crew.	Registration, Crew Management	Crew contains both racers; racer crew field updated.	PASS.	No issues. Validates registration to crew link.
T2	Buy and equip part, then verify race speed eligibility.	Registration, Inventory	Initially fails speed check; passes after upgrade; balance deducted correctly.	PASS.	No issues. Confirms inventory data affects eligibility checks.
T3	Register racer into race with entry fee and speed requirement checks.	Registration, Inventory, Race Management	Eligible racer enters race and pays fee; ineligible (slow) racer rejected.	PASS.	No issues. Confirms multi-condition race admission.

ID	Scenario Tested	Modules Involved	Expected Result	Actual Result	Errors / Issues Found
T4	Mission progress and bonus after race win.	Registration, Race Management, Results, Mission Planning	Winner should receive race prize and mission bonus.	FAIL.	`ValueError: No participants in race Race 1`. Root cause: inventory setup missing, so race registration returned False silently and no participant entered.
T5	Betting payout after race result.	Registration, Race Management, Results, Betting	Correct prediction should pay 2x bet amount.	FAIL.	`ValueError: No participants in race Big Race`. Root cause: inventory setup missing for racers used in race registration.
T6	Reputation update after race result.	Registration, Race Management, Results, Reputation	Winner +10 reputation, loser -2 from base 50.	FAIL.	`ValueError: No participants in race Street Cup`. Root cause: inventory setup missing before registration into race.
T7	Full end-to-end flow across all 8 modules.	All modules	Correct balances, mission behavior, betting payout, and reputation updates.	PASS.	No issues. This test includes correct inventory setup, so race participants are valid.

6 Why Each Test Is Needed (Simple Explanation)

- T1 ensures the system does not treat modules as isolated; registered racers must be transferable to team structures.
- T2 ensures inventory upgrades are not cosmetic and directly affect race eligibility.
- T3 ensures race entry uses combined checks (identity, money, and car readiness).
- T4 ensures race outcomes truly propagate into mission progress and financial rewards.
- T5 ensures post-race financial side effects are integrated, not manually mocked.
- T6 ensures social progression (reputation) is tied to actual race outcomes.
- T7 ensures the complete workflow remains consistent when all modules are used together.

7 Execution Results Summary

Latest execution of the integration suite produced:

- **4 passed**
- **3 failed**

The failing tests are integration defects in test setup sequence rather than failures in result, mission, betting, or reputation logic itself. Specifically, race registration depends on inventory initialization. If `setup_inventory` is not called for a racer, race registration returns `False` and the race has no participants, causing results calculation to raise an exception.

8 Detected Logical Issues and Fix Recommendations

8.1 Issue 1: Hidden dependency on inventory setup

Observed in: T4, T5, T6.

Problem: Tests attempted race registration without ensuring racer inventory is initialized.

Impact: Race participants list remains empty; downstream result processing fails.

Recommended fix options:

1. Update tests to call `InventoryModule.setup_inventory(racer_id, car)` before race registration.
2. Strengthen `register_for_race` by returning descriptive error objects/messages instead of only `False`, making setup issues visible.
3. Add assertion in tests immediately after `register_for_race` to verify successful entry before calculating race outcome.

8.2 Issue 2: Assignment rule traceability

Observation: The assignment describes role-based constraints (for example, only drivers race; mission role checks), while current implementation models racer capability through speed/inventory and mission enrollment.

Recommendation: Explicitly document this abstraction in the report and, if required by assessor, extend data model to role-tag crew members and enforce role checks in race and mission modules.

9 Data Flow Validation

The following cross-module data transitions were validated by passing tests and partial flows:

- Registration data (racer object) is reused by Crew, Race, Mission, Betting, and Reputation.
- Inventory modifies effective car speed used by Race Management gatekeeping.
- Race result updates racer balance through prize allocation.
- Mission module receives result callback and can award completion bonus.
- Betting and reputation modules consume race result object as a shared integration contract.

10 Conclusion

Phase 2 integration demonstrates that most module interactions are correctly wired and function as intended in end-to-end conditions (especially validated by T7). The major discovered defects are setup-order integration issues in three tests, where inventory initialization was missing before race registration. After applying the recommended setup fixes, the suite is expected to validate full integration behavior with stronger reliability and clearer failure diagnostics.

Appendix A: Raw Test Outcome Snapshot

- Command: `python -m pytest integration/tests/test_integration.py -q`
- Summary: `3 failed, 4 passed`
- Failed tests:
  - `test_mission_completion_via_results`
  - `test_custom_betting_module`
  - `test_custom_reputation_module`